Malicious Code Analysis : CMU Heinz College 95-759
Lab #1 : Debugging 101

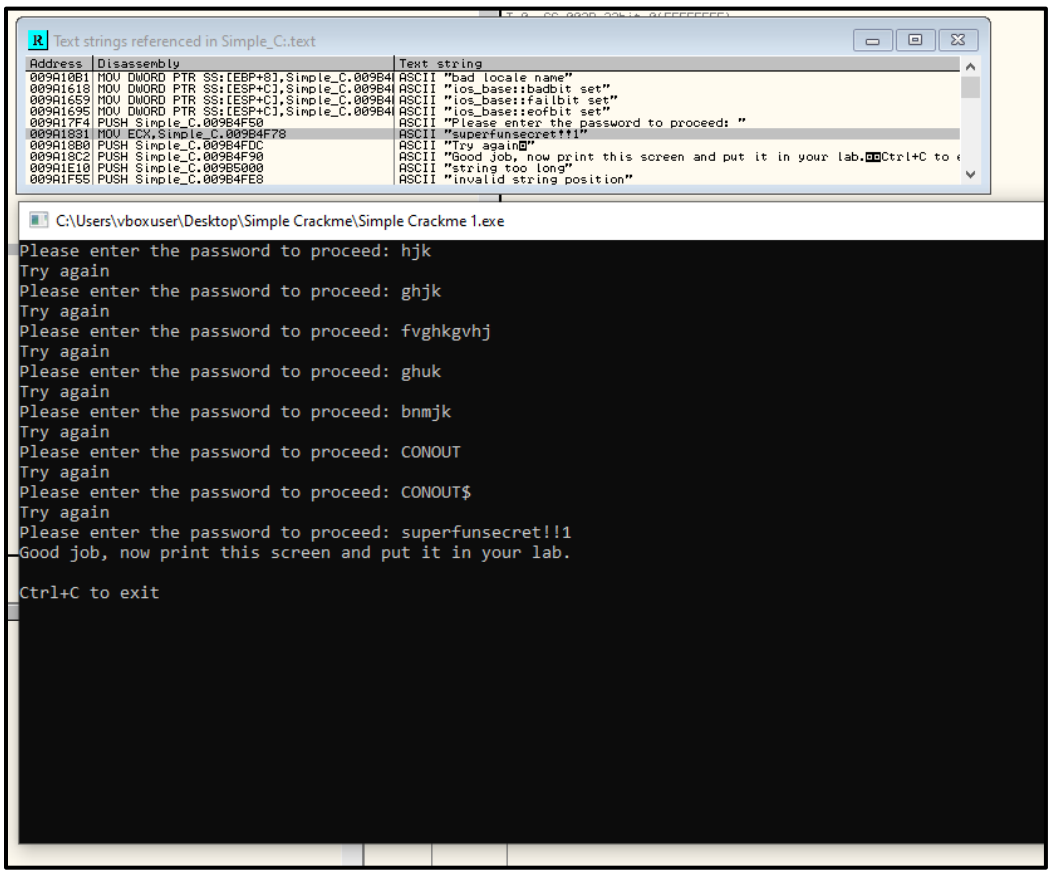
Name: Brendan R.
Date: 29 January 2026

Challenges:

Section 1 : Debugging Using OllyDbg

Use OllyDbg to find the password for the console application entitled 'Simple Crackme 1'. Select 'Run' (press F9) to start the program. Review the program and figure out where t
o set breakpoints at strategic locations to crack the password the application is looking for.

1. Paste the screen shot of the success window below:



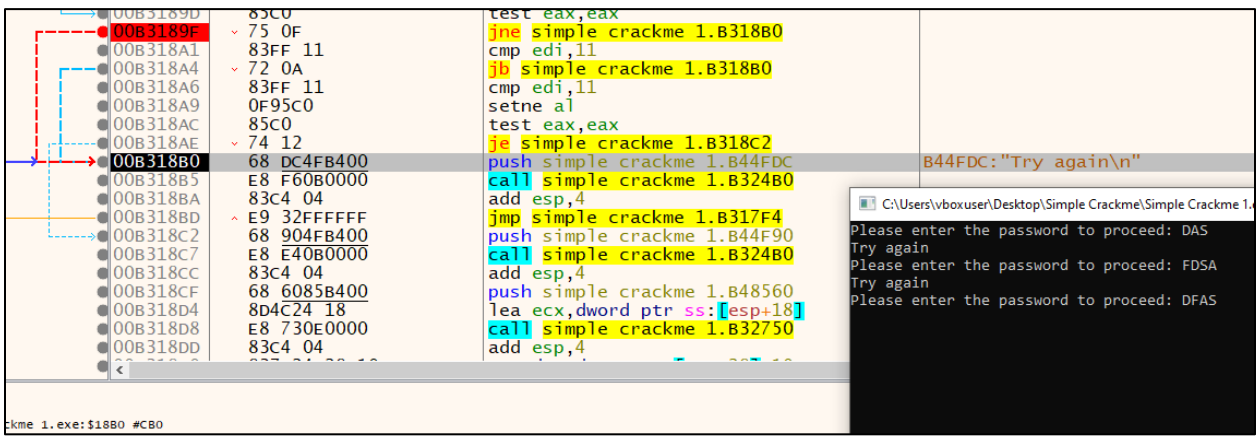
2. Explain your methodology in how you were able to locate the password **using your debugger**.

During my research and beginning to understand the program, I found the option to search for "All referenced strings". This revealed the portions of the code which held the plaintext strings of `simple_crackme.exe`. I did find it in the assembly code window of the debugger as well. TA's confirmed this method satisfies the expectation.

Fox, N. (2023, March 23). *What is x64dbg + how to use it*. Varonis. https://www.varonis.com/blog/how-to-use-x64dbg

YouTube. (n.d.). *Reverse Engineering with Ollydbg | HackTheBox "Find The Easy Pass" CTF Walkthrough*. YouTube. https://www.youtube.com/watch?v=GdzkRwoizFw

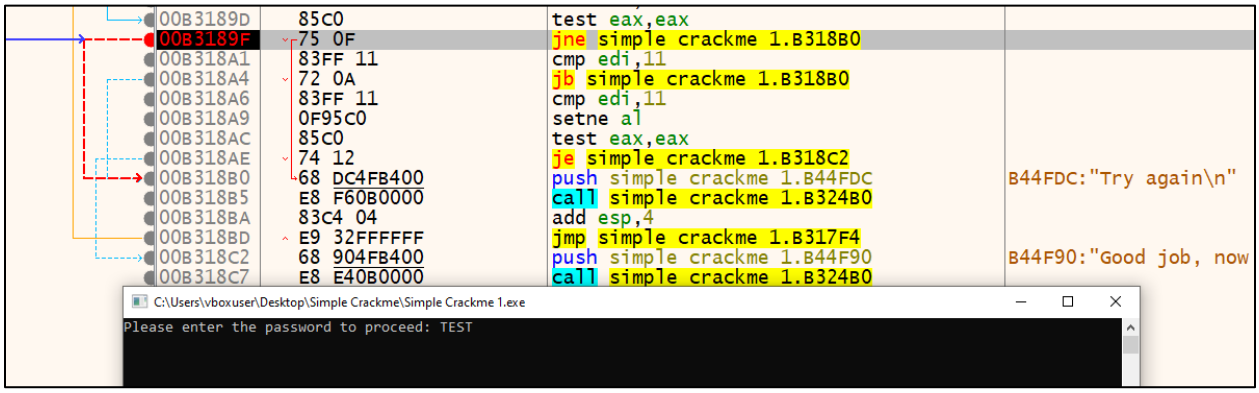
3. Create a breakpoint near the password comparison routine, then take a screenshot of the routine in Olly.

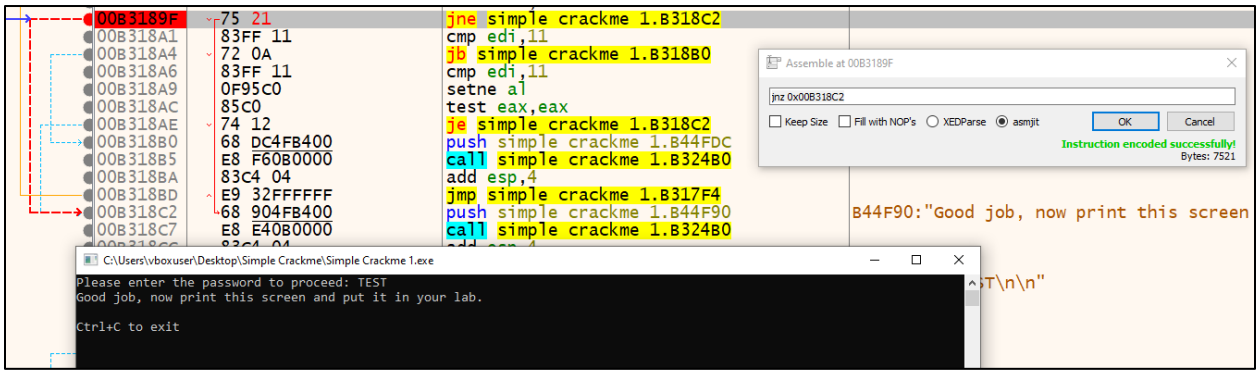


Above is the first breakpoint I set which confirmed to me the lines I was seeing was the password comparison routine. More on that in the writeup below.

4. How could you use your debugger to trick the application into thinking it received the right password (regardless of input)? Explain how in as detailed a manner as possible and provide screenshots to support your answer.

My initial thought to complete this challenge by getting to the success screen without entering the right password began by working backwards from the section that includes “Success” and “Try again”. Using x32dbg I located the end of the simple_crackme.exe file and found the Try Again line. I followed the flow of this line to the above line which CALLs it. I set a breakpoint here to see what would happen. The program then halted after entering a password and hitting the breakpoint. So, I knew the decision point of the program had to be around this line. However, nothing stood out to me as a line I could nullify to skip the comparison completely and end up at SUCCESS. So, I instead turned my attention to the arrow which connects this line 00B3189F to TRY AGAIN. I thought I could change the location this CALLs from TRY AGAIN to SUCCESS. That’s what I did and managed to **change the original 0x00B318B0 to 0x00B318C2**. I watched the arrow change and running the program produced a success.





Now that I knew the few lines where the comparison was taking place, I began exploring other ways to get the desired result without “brute-forcing” it like above. I landed on the result below which I will explain in further detail.

```
00B31899   EB 02        jmp simple crackme 1.B3189D
00B3189B   33C0         xor eax,eax
00B3189D   85C0         test eax,eax
00B3189F   90           nop
00B318A0   90           nop
00B318A1   83FF 11      cmp edi,11
00B318A4   90           nop
00B318A5   90           nop
00B318A6   83FF 11      cmp edi,11
00B318A9   0F95C0       setne al
00B318AC   85C0         test eax,eax
00B318AE   75 12        jne simple crackme 1.B318C2
00B318B0   68 DC4FB400  push simple crackme 1.B44FDC      B44FDC:"Try again\n"
00B318B5   E8 F60B0000  call simple crackme 1.B324B0
00B318BA   83C4 04      add esp,4
00B318BD   E9 32FFFFFF  jmp simple crackme 1.B317F4
00B318C2   68 904FB400  push simple crackme 1.B44F90      B44F90:"Good job, now print this
00B318C7   E8 E40B0000  call simple crackme 1.B324B0
```

```
C:\Users\vboxuser\Desktop\Simple Crackme\Simple Crackme 1.exe
Please enter the password to proceed: GFD
Try again
Please enter the password to proceed:
FDAS
Try again
Please enter the password to proceed: FDSA
Try again
Please enter the password to proceed: FDSA
Try again
Please enter the password to proceed: FDAS
Try again
Please enter the password to proceed: fdsa
Good job, now print this screen and put it in your lab.

Ctrl+C to exit
```

Because I knew the first line was the comparator that is hit first, I began exploring what could be done about that. I recalled hearing and reading about NOP and the fact that it is essentially “Do not operate.” I thought this may have a place in this program logic of trying to break this flow. I entered a NOP on this first line. However, even with this, I was still hitting the TRY AGAIN. I began to look further.

Again, looking at the arrows on the left of the assembly code, I noticed a second line pointing to TRY AGAIN:

```
00B318A4 | 72 0A | jb simple crackme 1.B318B0 |
```

I placed a breakpoint here and saw the program halt again! Changed this to NOP. However, it is still falling to TRY AGAIN. There are no more CALLs to that line, so I was struggling to come up with an explanation for this. After some research and trial and error, I noticed a line connected to the SUCCESS field:

```
00B318AE | 74 12 | je simple crackme 1.B318C2 |
```

The line connecting this was directly above TRY AGAIN. Maybe there’s something I can change on this line to skip ahead to GOOD JOB which it was connected to.

```
00B3189F   90           nop
00B318A0   90           nop
00B318A1   83FF 11      cmp edi,11
00B318A4   90           nop
00B318A5   90           nop
00B318A6   83FF 11      cmp edi,11
00B318A9   0F95C0       setne al
00B318AC   85C0         test eax,eax
00B318AE   74 12        je simple crackme 1.B318C2
00B318B0   68 DC4FB400  push simple crackme 1.B44FDC      B44FDC:"Try again\n"
00B318B5   E8 F60B0000  call simple crackme 1.B324B0
00B318BA   83C4 04      add esp,4
00B318BD   E9 32FFFFFF  jmp simple crackme 1.B317F4
00B318C2   68 904FB400  push simple crackme 1.B44F90      B44F90:"Good job, now print this
00B318C7   E8 E40B0000  call simple crackme 1.B324B0
```

This is where I got stuck for a bit. I finally revisited the SecureLayer7 resource and found within this article the section related to jumps. I replicated their solution mentioned here:

"Change JE SHORT Password.00457728 to JNE SHORT Password.00457728, JNE(Jump If Not Equal) means that if the password is correct it will give you the bad message and if the password is incorrect it will give you the correct message."

(Original) `00B318AE | 74 12 | je simple crackme 1.B318C2 |`

(New) `00B318AE | 75 12 | jne simple crackme 1.B318C2 |`

This small change worked! My understanding of the difference between **`je`** and **`jne`** is essentially:

```
je   = =  "password"
 jne !=  "password"
```

However, I also tested this method while omitting the NOPs in the lines above, and this change alone does not produce a success. I am confident this solution is more in the spirit of the exercise.